

10장. 파일 입출력

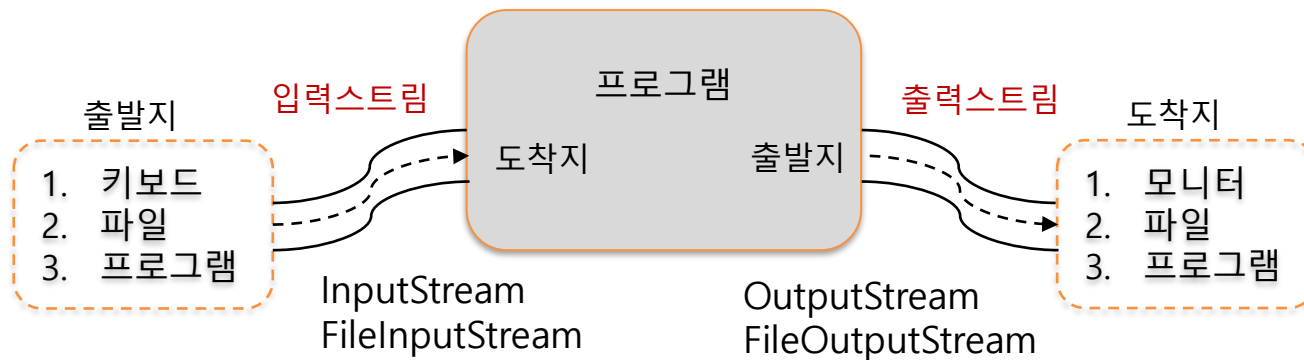


file IO



입, 출력 스트림

- **스트림**이란? 자료흐름이 물의 흐름과 같다는 뜻이다. 입출력 장치는 매우 다양하기 때문에 프로그램 호환성이 떨어짐
- 입출력 장치와 무관하고 일관성 있게 프로그램을 구현할 수 있도록 일종의 가상통로인 스트림을 제공
- 자료를 읽어 들이려는 소스(source)와 자료를 쓰려는 대상(target)에 따라 각각 다른 **스트림 클래스**를 제공
 - ✓ 입력 스트림 – 어떤 동영상을 재생하기 위해 동영상 파일에서 자료를 읽을때 사용함
 - ✓ 출력 스트림 – 편집 화면에 사용자가 쓴 글을 파일에 저장할 때는 출력 스트림 사용함



입, 출력 스트림

바이트 단위 스트림과 문자 단위 스트림

- 바이트 단위 스트림 - 그림, 동영상, 음악 파일등 대부분 파일은 바이트 단위로 읽거나 쓴다.
- 문자 단위 스트림 - 문자만 받고 보낼 수 있도록 특화되어 있다.

구분	바이트 기반 스트림		문자 기반 스트림	
	입력 스트림	출력 스트림	입력 스트림	출력 스트림
최상위 클래스	InputStream	OutputStream	Reader	Writer
하위 클래스	FileInputStream	FileOutputStream	FileReader	FileWriter



표준 입출력

● 표준 입출력

- **System** 클래스는 3개의 변수를 가지고 있는데, System.out은 표준 출력용, System.in은 표준 입력용 , 빨간색으로 오류를 표시할 때는 System.err을 사용한다.
- out, in, err 모두 정적 변수이다.
- 그 외 java.util 패키지에 있는 **Scanner** 클래스 – 문자, 정수, 실수 등을 읽을 수 있다.

자료형	변수 이름	설명
<code>static</code> PrintStream	out	표준 출력 스트림
<code>static</code> InputStream	in	표준 입력 스트림
<code>static</code> OutputStream	err	표준 오류 출력 스트림



표준 입출력

- System.in으로 화면에서 문자 1개 입력 받기

```
public class SystemInTest1 {  
    public static void main(String[] args) {  
        System.out.println("한 문자를 입력하고 [Enter]를 누르세요");  
        try {  
            int data = System.in.read();  
            System.out.println((char)data);  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

한 문자를 입력하고 [Enter]를 누르세요

b
b



표준 입출력

● System.in으로 화면에서 문자 여러개 입력 받기

```
public class SystemInTest2 {  
    public static void main(String[] args) {  
  
        System.out.println("여러 개의 문자를 입력하고 [Enter]를 누르세요.");  
  
        try {  
            int data;  
            while((data = System.in.read()) != -1) {  
                System.out.print((char)data);  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
  
        /*while(true) {  
            try {  
                int data = System.in.read();  
                if(data == -1) break;  
                System.out.print((char)data);  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
        */  
    }  
}
```

여러 개의 문자를 입력하고 [Enter]를 누르세요.

java

java



문자 단위 스트림 – Writer

■ Writer 클래스

- 주요 하위 클래스

스트림 클래스	설명
FileWriter	문자 단위로 파일에 자료를 씁니다.
OutputStreamWriter	파일에 바이트 단위로 출력한 자료를 문자로 변환해 주는 보조 스트림이다.
BufferedWriter	문자로 쓸 때 배열을 제공하여 한꺼번에 쓸 수 있는 기능을 제공해 주는 보조 스트림이다.

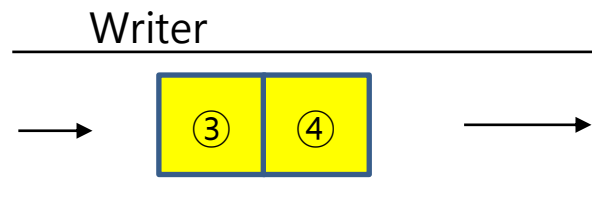
■ 주요 메소드

리턴타입	메소드	설명
void	write(int c)	한 문자를 파일에 출력한다.
void	write(char[] b)	문자 배열 buf의 내용을 파일에 출력한다.
void	flush()	파일에 출력하기 전에 자료가 있는 공간(출력버퍼)을 비워 출력
void	close()	출력 스트림과 연결된 대상 리소스를 닫는다.

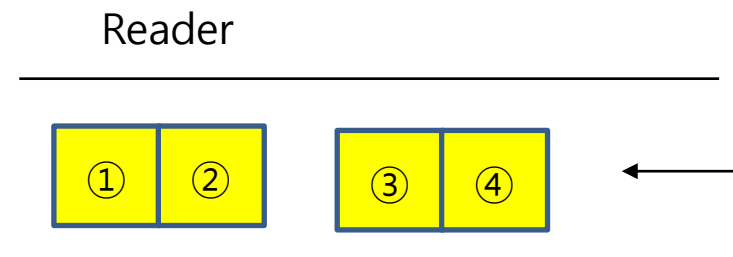
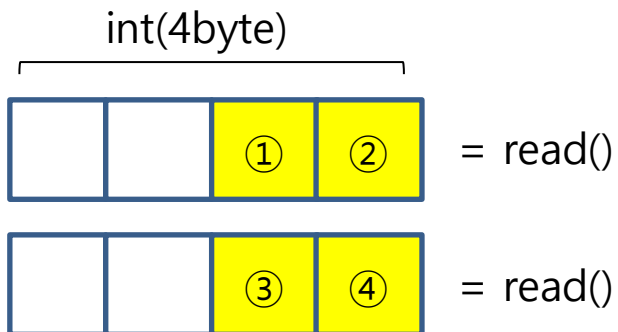


문자 단위 스트림 – Writer

- FileWriter 클래스로 문자 쓰기



- FileReader 클래스로 읽기



문자 단위 스트림 – Writer

- **FileWriter 클래스로 문자 쓰기**

- ✓ **파일을 이용한 입출력 과정**

1. 파일을 연다(open) -> 파일 경로에 파일 생성
2. 파일을 쓴다. -> 문자(열)를 쓴다(저장)
3. 파일을 닫는다. -> `writer.close()`



문자 단위 스트림 – Writer

■ 문자 쓰기

```
public class FileWriterTest {  
    public static void main(String[] args) {  
  
        try {  
            //파일 열기(file 디렉터리는 미리 생성하고, test.txt는 없으면 생성됨)  
            Writer writer = new FileWriter("c:/file/text.txt");  
  
            //한 개의 문자 쓰기  
            writer.write('A');  
            writer.write('b');  
  
            //숫자 쓰기 - 아스키 코드로 저장  
            writer.write(49);  
  
            //배열을 활용하여 문자 쓰기  
            char[] arr = {'C', 'D', 'E'};  
            writer.write(arr);  
        }  
    }  
}
```



문자 단위 스트림 – Writer

- 문자 쓰기

```
        writer.write('\n');    //줄바꿈

        //문자열 쓰기
        writer.write("apple");
        writer.write("좋아요");

        writer.flush();    //버퍼의 데이터 강제 쓰기

        //파일 닫기
        writer.close();

        System.out.println("파일 쓰기 완료!");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Ab1CDE
apple좋아요



문자 단위 스트림 – Reader

▪ Reader 클래스

- 주요 하위 클래스

스트림 클래스	설명
FileReader	파일에서 문자 단위(2바이트)로 읽는 스트림 클래스이다.
InputStreamReader	바이트 단위로 읽은 자료를 문자로 변환해 주는 보조 스트림
BufferedReader	문자로 읽을 때 배열을 제공하여 한꺼번에 읽을 수 있는 기능을 제공해 주는 보조 스트림이다.

▪ 주요 메소드

리턴타입	메소드	설명
int	read()	파일로부터 한 문자를 읽는다.
int	read(char[] buf)	파일로부터 buf 배열에 문자를 읽는다.
void	close()	스트림과 연결된 파일 리소스를 닫는다.



문자 단위 스트림 – Reader

- **FileReader** 클래스로 문자 쓰기

- ✓ 파일을 이용한 입출력 과정

1. 파일을 연다(open) -> 파일 경로에 파일을 연다
2. 파일을 읽는다. -> 문자(열)을 읽는다.
3. 파일을 닫는다. -> `reader.close()`



문자 단위 스트림 – Reader

- FileReader 클래스로 읽기

```
public class FileReaderTest {  
    public static void main(String[] args) {  
  
        Reader reader = null;  
        try {  
            //문자로 읽기  
            reader = new FileReader("c:/file/text.txt");  
            while(true) {  
                int data = reader.read();  
                if(data == -1) break;  
                System.out.print((char)data);  
            }  
            reader.close();  
            System.out.println();  
        }  
    }  
}
```



문자 단위 스트림 – Reader

- 파일에서 읽기

```
//문자 배열로 읽기
reader = new FileReader("c:/file/text.txt");
char[] data = new char[100];
while(true) {
    int num = reader.read(data);
    if(num == -1) break;
    for(int i = 0; i < num; i++) {
        System.out.print(data[i]);
    }
}
reader.close();
} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
}
```

Ab1CDE
apple좋아요
Ab1CDE
apple좋아요



문자 단위 스트림 – FileWriter

- 파일에 추가로 쓰기

```
public class FileWriterTest2 {  
    public static void main(String[] args) {  
  
        try {  
            // 데이터 추가(append)로 쓰기 - true 모드  
            Writer writer = new FileWriter("c:/file/text.txt", true);  
  
            writer.write("\n행운을 빌어요!!");  
  
            writer.close();  
  
            System.out.println("쓰기 완료!");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Ab1CDE
apple좋아요
행운을 빌어요!!



File 클래스

- File 클래스

디렉토리 생성, 파일 생성, 파일 정보 조회, 파일 삭제 등의 기본적인 파일 시스템 작업을 수행하는 클래스이다

```
public class FileTest {  
  
    public static void main(String[] args) throws IOException {  
  
        //파일 쓰기 전 디렉터리 존재 유무 확인  
        File dir = new File("c:/file2");  
        if (!dir.exists())  
            dir.mkdirs();    //디렉터리 생성  
  
        File file = new File("C:/file2/new_file.txt");  
        file.createNewFile();  
  
        System.out.println(file.isFile());  
        System.out.println(file.isDirectory());  
        System.out.println(file.getName());  
        System.out.println(file.getPath());  
  
        //file.delete();    //파일 삭제  
  
    }  
}
```

```
true  
false  
new_file.txt  
C:\file2\new_file.txt
```



바이트 단위 스트림 – OutputStream

➤ OutputStream 클래스

▪ 주요 하위 클래스

스트림 클래스	설명
FileOutputStream	바이트 단위로 파일에 자료를 씁니다.
BufferedOutputStream	기반 스트림에서 자료를 쓸때 추가 기능을 제공하는 보조 스트림의 상위 클래스이다.

▪ 주요 메소드

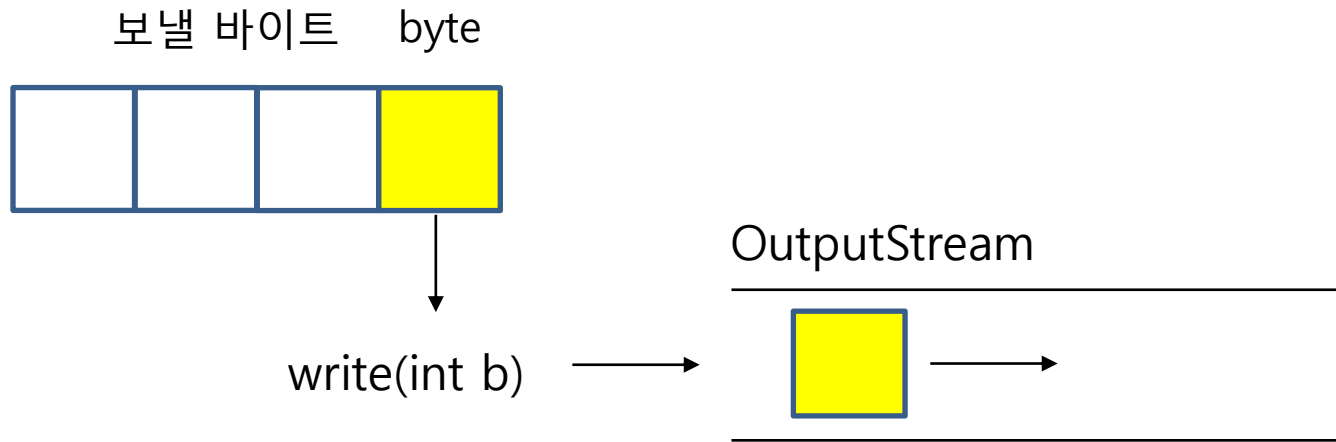
리턴타입	메소드	설명
void	write(int b)	한 바이트를 출력한다.
void	write(byte[] b)	b[] 배열에 있는 자료를 출력한다.
void	write(byte b[], int off, int len)	b[] 배열에 off 위치부터 len개수 만큼 출력
void	close()	출력 스트림과 연결된 대상 리소스를 닫는다.



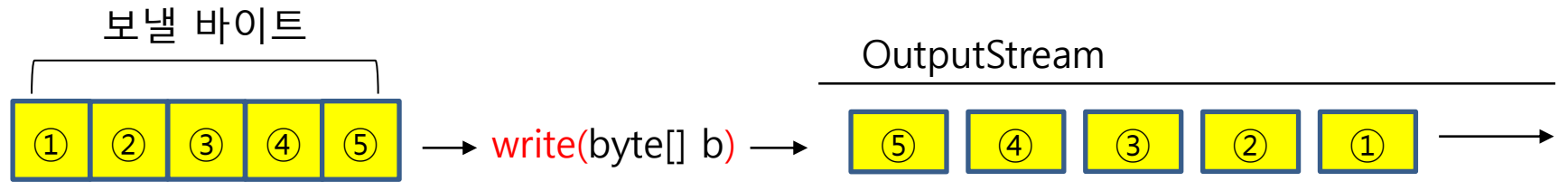
FileOutputStream

➤ 파일에 바이트 자료 쓰기

write(int b) 메서드는 매개값 int(4byte)에서 끝 1byte만 출력한다.



➤ write(byte[] b) – 파일에 바이트 배열로 쓰기



FileOutputStream

➤ 파일에 바이트 자료 쓰기

```
public class OutputStreamTest1 {  
    public static void main(String[] args) {  
        try {  
            //바이트 출력 스트림 생성  
            OutputStream os = new FileOutputStream("C:/file/file1.bin");  
            byte a = 10;  
            byte b = 20;  
  
            os.write(a); // 바이트 값 10 (0x0A) 기록  
            os.write(b); // 바이트 값 20 (0x14) 기록  
  
            //배열로 쓰기  
            byte[] array = {65, 66, 67};  
            os.write(array);  
            //os.write(new byte[]{65, 66, 67}); // "ABC" (ASCII 코드)  
  
            os.flush();  
            os.close();  
            System.out.println("쓰기 완료!!");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



FileOutputStream

➤ 파일에 바이트 자료 쓰기

```
public class OutputStreamTest2 {  
  
    public static void main(String[] args) {  
        //try ~ with ~ resource문 : close() 하지 않아도 됨  
        try(OutputStream os = new FileOutputStream("c:/file/file2.bin")){  
            //특정 데이터 보내기  
            byte[] array = {48, 49, 50, 51, 52};  
  
            os.write(array, 1, 3); //1번 인덱스 부터 3개 쓰기  
  
            //문자열로 변환하여 쓰기  
            String text = "\nHava a nice day!";  
            byte[] textBytes = text.getBytes();  
            os.write(textBytes);  
  
            os.flush(); //버퍼에 강제 쓰기  
            System.out.println("쓰기 완료!");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



바이트 단위 스트림 – InputStream

➤ InputStream

▪ 주요 하위 클래스

스트림 클래스	설명
FileInputStream	파일에서 바이트 단위로 자료를 읽는다.
BufferedInputStream	기본 스트림에서 자료를 읽을때 추가 기능을 제공하는 보조 스트림의 상위 클래스이다.

▪ 주요 메소드

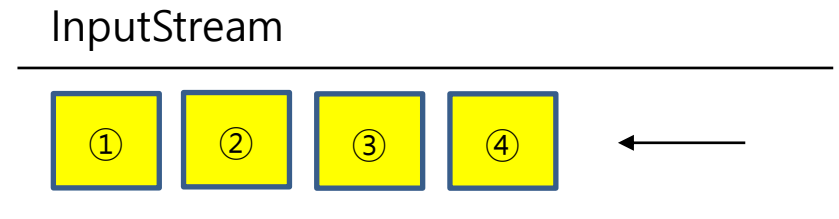
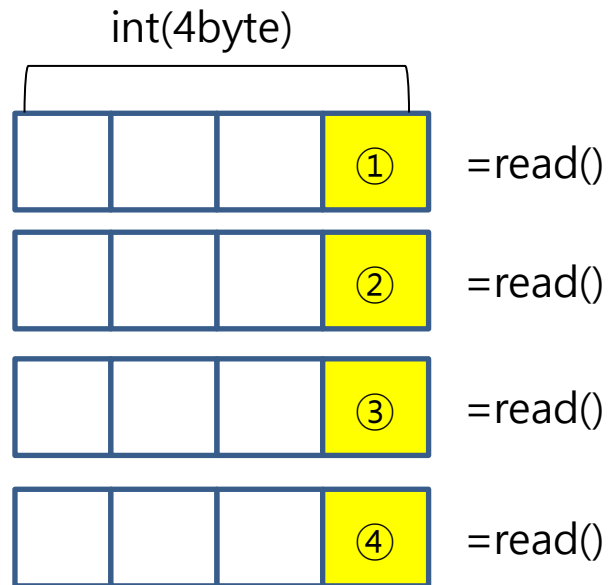
리턴타입	메소드	설명
int	read()	입력 스트림으로부터 1바이트를 읽고 읽은 바이트를 리턴한다.
int	read(byte[] b)	입력 스트림으로부터 읽은 바이트들을 매개값으로 주어진 바이트 배열 b에 저장하고 실제로 읽은 바이트 수를 리턴한다.
void	close()	사용한 시스템 자원을 반납하고 입력 스트림을 닫는다.



FileInputStream

➤ 파일에서 자료 읽기 - 1byte

read() 메서드는 입력 스트림으로부터 1byte를 읽고 int(4byte) 타입으로 리턴한다.
따라서 리턴된 4byte 중 끝 1byte에만 데이터가 들어있음



FileInputStream

➤ 파일에서 자료 읽기

```
public class InputStreamTest1 {  
  
    public static void main(String[] args) {  
  
        try {  
            InputStream is = new FileInputStream("C:/file/file1.bin");  
            int data;  
            while((data = is.read()) != -1) {  
                System.out.println(data);  
                //System.out.println((char)data);  
            }  
            is.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



FileInputStream

➤ 파일에서 자료 읽기

```
public class InputStreamTest2 {  
    public static void main(String[] args) {  
        try(InputStream is = new FileInputStream("C:/file/file2.bin")){  
            byte[] data = new byte[100];  
  
            while(true) {  
                int num = is.read(data);  
                if(num == -1) break;  
                for(int i = 0; i < num; i++) {  
                    System.out.print((char)data[i]);  
                }  
            }  
        } catch(IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
123  
Hava a nice day!
```



이미지 파일 복사

- FileInputStream과 FileOutputStream을 이용하여 파일 복사하기



이미지 파일 복사

➤ FileInputStream과 FileOutputStream을 이용하여 파일 복사하기

```
public class FileCopyTest1 {  
    public static void main(String[] args) {  
        String originFile = "c:/file/boat.jpg";  
        String copyFile = "c:/file/boat2.jpg";  
        long start, end;  
  
        try(InputStream is = new FileInputStream(originFile);  
            OutputStream os = new FileOutputStream(copyFile)){  
            start = System.currentTimeMillis(); //시작 시각  
  
            while(true) {  
                int num = is.read();  
                if(num == -1) break;  
                os.write(num);  
            }  
            os.flush();  
            end = System.currentTimeMillis(); //종료 시각  
            System.out.println("복사 소요 시간: " + (end - start) + "ms");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

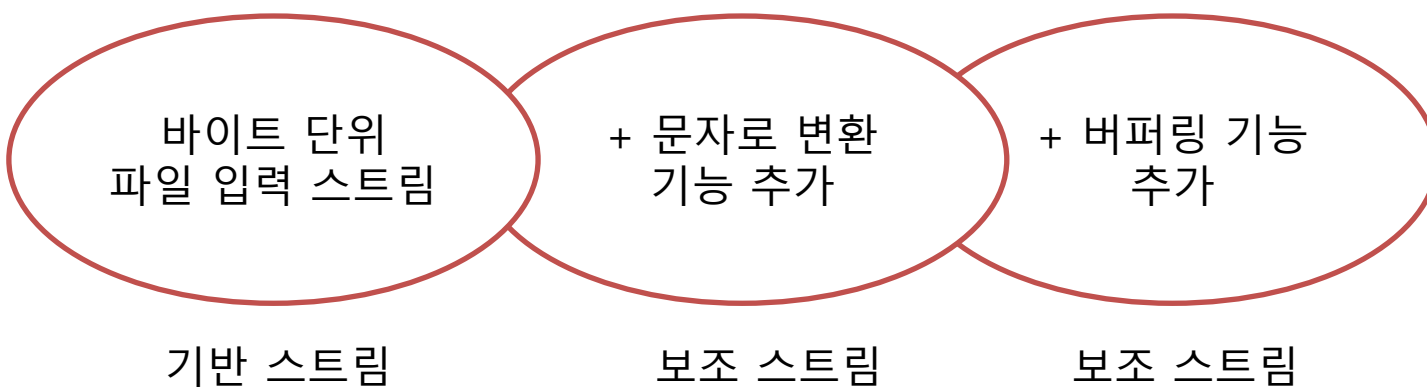
복사 소요 시간: 1556ms



보조 스트림

보조 스트림

- 다른 스트림과 연결되어 여러 가지 편리한 기능을 제공해주는 스트림을 말한다.
- 자체적으로 입출력을 수행할 수 없기 때문에 입출력 소스로부터 직접 생성된 입출력 스트림에 연결해서 사용함
- 생성자의 매개변수로 **기반스트림**을 가짐



보조 스트림 – InputStreamReader

InputStreamReader와 OutputStreamWriter

- 바이트 자료만 입력되는 스트림에서 문자로 변환해 준다.
- **System.in** 이나 네트워크 **socket** 통신을 할때 쓰인다.
- 입출력 기능이 없으므로 다른 입출력 스트림을 포함한다.

생성자	설명
InputStreamReader (FileInputStream in)	생성자의 매개변수로 FileInputStream을 받는다.
OutputStreamWriter (FileOutputStream out)	생성자의 매개변수로 FileOutputStream을 받는다.



보조 스트림 – InputStreamReader

● OutputStreamWriter & InputStreamReader

- ✓ 바이트 자료만 출력되는 스트림에서 문자로 변환해 파일을 생성하고 읽기
- ✓ 파일 쓰기 후 프로젝트 우클릭 -> Refresh(F5) -> memo.txt 표시

```
public class OutputStreamWriterTest {  
  
    public static void main(String[] args) {  
        //보조 스트림은 기반 스트림을 생성자로 함  
        //문자 쓰기  
        try(OutputStreamWriter osw =  
            new OutputStreamWriter(new FileOutputStream("memo.txt"))){  
            osw.write("오늘도 좋은 하루 되세요~");  
            System.out.println("memo.txt 파일을 열어 보세요~");  
        } catch (IOException e) {  
            System.out.println(e);  
        }  
    }  
}
```

> Chapter12 [korea_IT main]
> JRE System Library [JavaSE-21]
v > src
 > decorator
 > output_inputstream
 > systemin
 > writer_reader
memo.txt
message.txt
object.dat



보조 스트림 – InputStreamReader

- **OutputStreamWriter & InputStreamReader**

바이트 자료만 출력되는 스트림에서 문자로 변환해 파일을 생성하고 읽기

```
//문자 읽기
try(InputStreamReader isr =
    new InputStreamReader(new FileInputStream("memo.txt"))){
    int data;
    while((data = isr.read()) != -1) {
        System.out.print((char)data);
    }
}catch(IOException e) {
    System.out.println(e);
}
}
```



보조 스트림 – Buffered 스트림

● Buffered 스트림

- 입출력이 한 바이트나 문자 단위로 이루어지면 그만큼 프로그램 수행 속도가 느려진다.
- Buffered 스트림은 내부적으로 8,192바이트 크기의 배열을 가지고 있으며 더 빠르게 입출력을 수행하는 버퍼링 기능을 제공한다.

스트림 클래스	설명
BufferedInputStream (InputStream in)	바이트 단위로 읽는 스트림에 버퍼링 기능을 제공
BufferedOutputStream (FileOutputStream out)	바이트 단위로 출력하는 스트림에 버퍼링 기능을 제공
BufferedReader	문자 단위로 읽는 스트림에 버퍼링 기능을 제공
BufferedWriter	문자 단위로 출력하는 스트림에 버퍼링 기능을 제공



보조 스트림 – Buffered 스트림

● BufferedInputStream과 BufferedOutputStream으로 파일 복사하기

```
public class BufferedStreamTest {  
    public static void main(String[] args) {  
        long start, end; //시작, 종료시간  
        String originFile, copyFile;  
  
        originFile = "c:/file/boat.jpg"; //원본 파일  
        copyFile = "c:/file/boat2.jpg"; //복사 파일  
        //보조 스트림은 기반 스트림을 생성자의 매개변수로 갖는다.  
        try(FileInputStream fis = new FileInputStream(originFile);  
            FileOutputStream fos = new FileOutputStream(copyFile);  
            BufferedInputStream bis = new BufferedInputStream(fis);  
            BufferedOutputStream bos = new BufferedOutputStream(fos)){  
            start = System.currentTimeMillis();  
            int i;  
            while((i = bis.read()) != -1) {  
                bos.write(i);  
            }  
            end = System.currentTimeMillis();  
            System.out.println("파일 복사 소요시간: " + (end - start) + "ms");  
        }catch(IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

파일 복사 소요시간: 37ms



보조 스트림 – Buffered 스트림

● **BufferedReader**

- 문자 입력 스트림 Reader에 BufferedReader를 연결하면 행 단위로 문자열을 읽는 매우 편리한 `readLine()` 메소드를 제공한다.
- `readLine()`은 enter 키 이전의 모든 문자열을 읽고 리턴한다.

readLine

```
public String readLine() throws IOException
```

Reads a line of text. A line is considered to be terminated by any one of a line feed ('\n'), a carriage return ('\r'), a carriage return followed immediately by a line feed, or by reaching the end-of-file (EOF).

오늘도 좋은 하루되세요!!
감사합니다.
여름 장마비가 옵니다.



보조 스트림 – Buffered 스트림

- **BufferedReader**

```
public class WriteExample {  
    public static void main(String[] args) {  
  
        try(Writer writer = new FileWriter("message.txt")){  
  
            String message = "오늘도 좋은 하루 되세요\n행운을 빌어요\n감사합니다.";  
            writer.write(message);  
  
            writer.flush();  
            writer.close();  
  
            System.out.println("파일 쓰기 완료!");  
        }catch(IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



보조 스트림 – Buffered 스트림

- **BufferedReaderTest**

```
try(FileReader fr = new FileReader("message.txt");
    BufferedReader br = new BufferedReader(fr)){

    /*while(true) {
        String str = br.readLine(); //1행 읽음
        if(str == null) break;
        System.out.println(str);
    }*/
    String str;
    //읽은 행이 없을 때까지 루프(반복)
    while((str = br.readLine()) != null) {
        System.out.println(str);
    }
    br.close();
}catch(IOException e) {
    e.printStackTrace();
}
```

오늘도 좋은 하루 되세요
행운을 빌어요
감사합니다.



보조 스트림 – Buffered 스트림

● BufferedReaderTest

```
public class BufferedReaderTest2 {  
  
    public static void main(String[] args) {  
  
        try(BufferedReader br = new BufferedReader(  
            new FileReader("message.txt"))){  
            String str;          //파일에서 읽은 문자열을 저장할 변수  
            int lineNo = 1;      //행 번호  
            while((str = br.readLine()) != null) {  
                System.out.println(lineNo + " " + str);  
                lineNo++;  
            }  
        }catch(IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

1 오늘도 좋은 하루 되세요
2 행운을 빌어요
3 감사합니다.



보조 스트림 – BufferedReader

- split()를 함수를 사용하여 문자열 분리하기

```
public class WriteExample2 {  
    public static void main(String[] args) {  
  
        try(Writer writer = new FileWriter("word.txt")){  
  
            String words = "ant bear cow chicken eagle elephant "  
                + "fox horse monkey penguin tiger";  
            writer.write(words);  
  
            writer.flush();  
            System.out.println("파일 쓰기 완료!");  
        }catch(IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



보조 스트림 – BufferedReader

- split()를 함수를 사용하여 문자열 분리하기

```
try(BufferedReader br = new BufferedReader(  
    new FileReader("word.txt"))){  
    String str;           //읽은 문자열 저장  
    String[] list = null; //분리된 단어를 저장할 배열  
  
    while((str = br.readLine()) != null) {  
        list = str.split(" ");  
    }  
    System.out.println(list[0]); //배열의 첫번째 요소  
    System.out.println(Arrays.toString(list)); //배열 객체 출력  
  
    //랜덤 추출  
    int rndIdx = (int)(Math.random() * list.length); //난수 저장  
    System.out.println(list[rndIdx]);  
}catch(IOException e) {  
    e.printStackTrace();  
}
```

첫 단어: ant

[ant, bear, cow, chicken, eagle, elephant, fox, horse, monkey, penguin, tiger]

단어: chicken



Typing Game

● 타자 연습 게임 프로젝트 > 배포하기

```

v > TypingGame2 [korea_IT main]
  > JRE System Library [JavaSE-21]
  v > src
    v > typing
      > TypingGame.java
      > WriteExample.java
    > module-info.java
  word.txt

```

```
c:\java_jar>cd typing_game2
```

```
c:\java_jar\typing_game2>dir
```

C 드라이브의 볼륨에는 이름이 없습니다.
볼륨 일련 번호: 4A81-5207

c:\java_jar\typing_game2 디렉터리

```

2025-06-01 오전 07:19 <DIR> .
2025-06-01 오전 07:19 <DIR> ..
2025-06-01 오전 07:19          3,366 typing_game2.jar
2025-06-01 오전 07:02           66 word.txt
                2개 파일              3,432 바이트
                2개 디렉터리 31,275,671,552 바이트 남음

```

```
c:\java_jar\typing_game2>java -jar typing_game2.jar
[타자 게임], 준비되면 엔터!
```

```

문제 번호 1
elephant
elephant
통과!!

```



보조 스트림 – BufferedReader

```
public class TypingGame {
    public static void main(String[] args) {
        try(FileReader fr = new FileReader("word.txt");
            BufferedReader br = new BufferedReader(fr)){
            // word.txt 파일 읽기
            String str;           //읽은 문자열 저장
            String[] words = null; //분리된 단어를 저장할 배열

            while((str = br.readLine()) != null) {
                words = str.split(" ");
            }
            //System.out.println(words[0]);
            //영어 타자 게임 준비
            int n = 1; //문제 번호
            long start, end; //시작, 종료 시간
            double elapsedTime;

            Scanner sc = new Scanner(System.in);
            System.out.print("[타자 게임], 준비되면 엔터!");
            sc.nextLine();
            start = System.currentTimeMillis(); //시작시간
```



보조 스트림 – BufferedReader

```
while(n <= 10) {
    Random rnd = new Random();
    int rndIdx = rnd.nextInt(words.length);
    String question = words[rndIdx];
    System.out.println("\n문제 번호 " + n);
    System.out.println(question); //문제 출제

    String answer = sc.nextLine(); //사용자 입력
    if(question.equals(answer)) {
        System.out.println("통과!!");
        n++; //다음 문제 번호
    }else {
        System.out.println("오타! 다시 도전!");
    }
}
end = System.currentTimeMillis(); //종료시간

elapsedTime = (double)(end - start) / 1000;
//서식 문자 사용 : %d-정수, %f-실수, %s-문자열
System.out.printf("게임 소요시간 %.2f초", elapsedTime);
sc.close();
}catch(IOException e) {
    e.printStackTrace();
}
```



보조스트림 – DataStream

● DataInputStream과 DataOutputStream

지금까지의 공부한 스트림은 사람이 읽고 쓰는 텍스트 또는 이미지 형식의 자료를 다루었으나, DataStream은 메모리에 저장된 0, 1상태를 그대로 읽거나 쓴다. 즉, 자료형의 크기가 그대로 보존된다.

메서드	설명
byte readByte()	1바이트를 읽어 반환
char readChar()	한 문자를 읽어 반환
int readInt()	4바이트를 읽어 정수값 반환
double readDouble()	8바이트를 읽어 실수값 반환
String readUTF()	문자열을 읽어 반환

DataInputStream

DataOutputStream

메서드	설명
void writeByte(int v)	1바이트의 자료 쓰기
void writeChar(int v)	2바이트의 자료 쓰기
void writeInt(int v)	4바이트의 자료 쓰기
void writeDouble(double v)	8바이트의 자료 쓰기
void writeUTF(String str)	문자열을 쓰기



DataStream

● DataInputStream과 DataOutputStream

```
public class DataInputStreamTest {  
    public static void main(String[] args) {  
        //데이터 쓰기  
        try (FileOutputStream fos = new FileOutputStream("object.dat");  
             DataOutputStream dos = new DataOutputStream(fos)){  
  
            //기본 타입 출력  
            dos.writeInt(1);           //학번  
            dos.writeUTF("우영우");    //이름  
            dos.writeDouble(95.5);     //점수  
  
            dos.writeInt(2);           //학번  
            dos.writeUTF("장그래");    //이름  
            dos.writeDouble(85.7);     //점수  
  
            dos.flush();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



DataStream

● DataInputStream과 DataOutputStream

```
//데이터 읽기
try (FileInputStream fis = new FileInputStream("object.dat");
    DataInputStream dis = new DataInputStream(fis)){

    for(int i = 0; i < 2; i++) {
        int no = dis.readInt();
        String name = dis.readUTF();
        double score = dis.readDouble();
        System.out.println(no + ". " + name + " : " + score);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
```

1. 우영우	: 95.5
2. 장그래	: 85.7

